CS201 DSA PROJECT

Reimagine Sentiment Analysis for Airline Reviews Insights

Data Structures for Polarized Word-Frequency Counts

G2T3 - TEAM TOUCH GRASS

NOV 2025

Problem Statement

Information Overload

Difficult to Interpret Large Volumes of Text

Sentiment Differentiation

Words Distinguish Positive from Negative reviews?

Efficient Computation at Scale

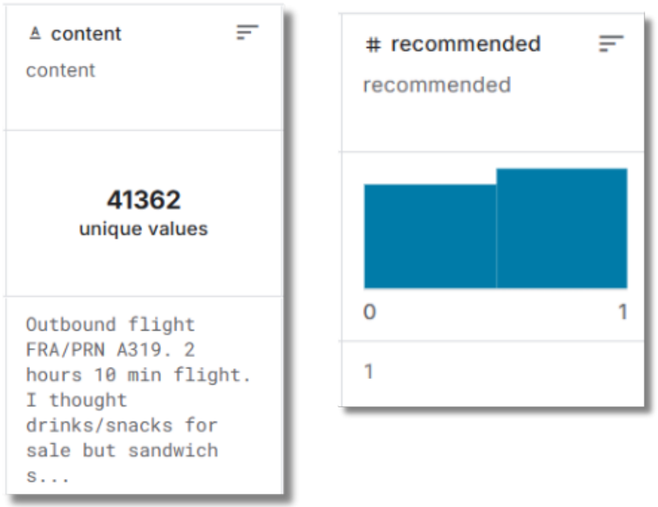
Slow and memory-heavy as Sentiment Data Scales

Exploring Data Structures in Sentiment Analysis

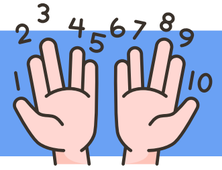
Data Tokenization Pre-Processing

SkyTrax dataset

- **Pronoun Filtering**
- **For a given airline, split reviews by their "recommended"**



Counting the Tokens/Words



- **Count Token Frequencies**
- **Extract Top-K per class**

```
[BST] Time (median of 5): 23 ms | Allocated (median): 330 KB
[GOOD] Top 10 most common words in POSITIVE reviews:
 1. Word: about           Count: 136
 ...
[BAD] Top 10 most common words in NEGATIVE reviews:
 1. Word: hour            Count: 679
 ...
```

- **Interactive Comparisons**

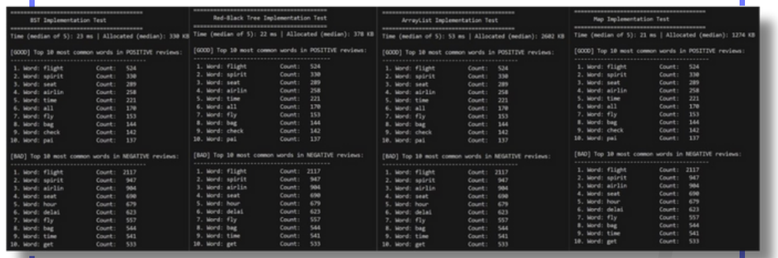
```
================================================================================
  COMPARISON RESULTS FOR: "DELAY"
  Sentiment: NEGATIVE reviews
  Total airlines with this word: 281
  Minimum reviews for reliability: 50
================================================================================

================================================================================
TOP 5 Airlines with HIGHEST weighted score for "delay":
(Weighted score considers both frequency % and review count)
================================================================================
Rank Airline                   Reviews     Count     Percent        Score
--------------------------------------------------------------------------------
1    sunwing-airlines          286         407       142.31%        349.78
2    allegiant-air             351         375       106.84%        272.07
3    spirit-airlines           786         623        79.26%        229.54
4    norwegian                 178         175        98.31%        221.49
5    flybe                     227         205        90.31%        212.94

BUILD SUCCESSFUL in 1m 8s
3 actionable tasks: 1 executed, 2 up-to-date
```

Experimental Results

- **Analyze Time Complexity and Space Complexity**



- **5x, 10x, 15x csv to measure asymptotic complexity**

```
# --- Configuration ---
# Set the path to your input CSV here. If your
input_csv = "airline.csv"  # <-- change this t

# Replication factors you want to output
factors = [10, 100, 1000]
```

Demo

```
========================================
      BST Implementation Test
========================================
Time (median of 5): 23 ms | Allocated (median): 330 KB

[GOOD] Top 10 most common words in POSITIVE reviews:
----------------------------------------
 1. Word: flight          Count:   524
 2. Word: spirit          Count:   330
 3. Word: seat            Count:   289
 4. Word: airlin          Count:   258
 5. Word: time            Count:   221
 6. Word: all             Count:   170
 7. Word: fly             Count:   153
 8. Word: bag             Count:   144
 9. Word: check           Count:   142
10. Word: pai             Count:   137

[BAD] Top 10 most common words in NEGATIVE reviews:
----------------------------------------
 1. Word: flight          Count:  2117
 2. Word: spirit          Count:   947
 3. Word: airlin          Count:   904
 4. Word: seat            Count:   690
 5. Word: hour            Count:   679
 6. Word: delai           Count:   623
 7. Word: fly             Count:   557
 8. Word: bag             Count:   544
 9. Word: time            Count:   541
10. Word: get             Count:   533
```

```
========================================
      Red-Black Tree Implementation Test
========================================
Time (median of 5): 22 ms | Allocated (median): 378 KB

[GOOD] Top 10 most common words in POSITIVE reviews:
----------------------------------------
 1. Word: flight          Count:   524
 2. Word: spirit          Count:   330
 3. Word: seat            Count:   289
 4. Word: airlin          Count:   258
 5. Word: time            Count:   221
 6. Word: all             Count:   170
 7. Word: fly             Count:   153
 8. Word: bag             Count:   144
 9. Word: check           Count:   142
10. Word: pai             Count:   137

[BAD] Top 10 most common words in NEGATIVE reviews:
----------------------------------------
 1. Word: flight          Count:  2117
 2. Word: spirit          Count:   947
 3. Word: airlin          Count:   904
 4. Word: seat            Count:   690
 5. Word: hour            Count:   679
 6. Word: delai           Count:   623
 7. Word: fly             Count:   557
 8. Word: bag             Count:   544
 9. Word: time            Count:   541
10. Word: get             Count:   533
```

```
========================================
      ArrayList Implementation Test
========================================
Time (median of 5): 53 ms | Allocated (median): 2602 KB

[GOOD] Top 10 most common words in POSITIVE reviews:
----------------------------------------
 1. Word: flight          Count:   524
 2. Word: spirit          Count:   330
 3. Word: seat            Count:   289
 4. Word: airlin          Count:   258
 5. Word: time            Count:   221
 6. Word: all             Count:   170
 7. Word: fly             Count:   153
 8. Word: bag             Count:   144
 9. Word: check           Count:   142
10. Word: pai             Count:   137

[BAD] Top 10 most common words in NEGATIVE reviews:
----------------------------------------
 1. Word: flight          Count:  2117
 2. Word: spirit          Count:   947
 3. Word: airlin          Count:   904
 4. Word: seat            Count:   690
 5. Word: hour            Count:   679
 6. Word: delai           Count:   623
 7. Word: fly             Count:   557
 8. Word: bag             Count:   544
 9. Word: time            Count:   541
10. Word: get             Count:   533
```

```
========================================
      Map Implementation Test
========================================
Time (median of 5): 21 ms | Allocated (median): 1274 KB

[GOOD] Top 10 most common words in POSITIVE reviews:
----------------------------------------
 1. Word: flight          Count:   524
 2. Word: spirit          Count:   330
 3. Word: seat            Count:   289
 4. Word: airlin          Count:   258
 5. Word: time            Count:   221
 6. Word: all             Count:   170
 7. Word: fly             Count:   153
 8. Word: bag             Count:   144
 9. Word: check           Count:   142
10. Word: pai             Count:   137

[BAD] Top 10 most common words in NEGATIVE reviews:
----------------------------------------
 1. Word: flight          Count:  2117
 2. Word: spirit          Count:   947
 3. Word: airlin          Count:   904
 4. Word: seat            Count:   690
 5. Word: hour            Count:   679
 6. Word: delai           Count:   623
 7. Word: fly             Count:   557
 8. Word: bag             Count:   544
 9. Word: time            Count:   541
10. Word: get             Count:   533
```

Array List

1. Collect reviews: pick the airline's reviews. If none, return two empty lists.

2. Split words: scan every review: if it's "recommended," add tokens to goodWords; else to badWords.

3. Count words: turn goodWords and badWords into (word, count) lists (compressCounts).

4. Rank by frequency: sort each (word, count) list from highest count to lowest.

5. Take the top 10: keep the first 10 from each list and return (topGood, topBad).

```
private HashMap<String, List<AirlineReview>> airlineReviews;
private String airline;

public AirlineArrayListImpl(HashMap<String, List<AirlineReview>> airlineReviews, String airline) {
    ...
}

public Pair<List<WordCount>, List<WordCount>> getTop10MostCommonWords() {
    ...
    return Pair.<List<WordCount>, List<WordCount>>of(topGood, topBad);
}

private List<WordCount> topKFromArrayList(List<WordCount> counts, int k) {
    ArrayList<WordCount> sorted = new ArrayList<>(counts);
    sorted.sort((a, b) -> Integer.compare(b.getCount(), a.getCount()));

    ArrayList<WordCount> result = new ArrayList<>();
    int limit = Math.min(k, sorted.size());
    for (int idx = 0; idx < limit; idx++) {
        result.add(sorted.get(idx));
    }

    return result;
}
```

Array List

Time Complexity:

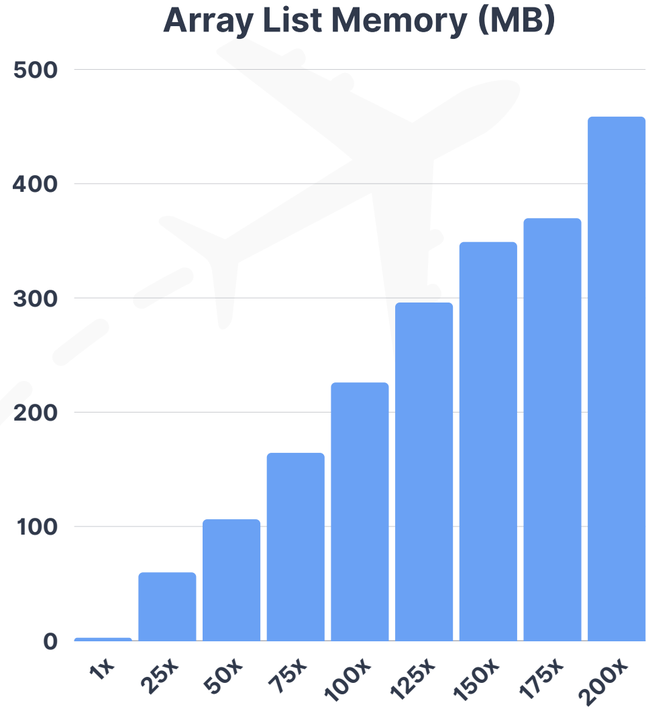
- O(N x M x log(N x M))

Space Complexity:

- O(N x M)

Where:

- N = Number of reviews
- M = Average words per review



Empirical Results	1x	25x	50x	75x	100x	125x	150x	175x	200x
Runtime (ms)	38	627	1265	1816	2534	3149	3680	4556	5229
Memory (MB)	2.602	59.811	106.294	164.428	225.750	296.118	348.732	369.768	458.685

Map

1. Grab reviews: get this airline's reviews; if none, return two empty top-10 lists.

2. Count with maps: use two hash maps (freqGood, freqBad) to tally how many times each word appears in good vs bad reviews.

3. Bucket & rank: group words by their counts using a TreeMap in reverse order (big counts first).

4. Take top 10: walk the TreeMap from highest count down and collect the first 10 words for each side.

5. Return results: output topGood and topBad as lists of (word, count).

```
private Map<String, List<AirlineReview>> airlineReviews;
private String airline;

public AirlineMapImpl(Map<String, List<AirlineReview>> airlineReviews, String airline) {
    ...
    return Pair.<List<WordCount>, List<WordCount>>of(topGood, topBad);

}

// Same logic as MapImplementationTest, reformatted to match other implementations
public Pair<List<WordCount>, List<WordCount>> getTop10MostCommonWords() {
    ...
    return Pair.of(goodList, badList);
}

// Map-specific Top-K using TreeMap bucketing (keeps original selection logic)
private List<WordCount> topKFromMap(HashMap<String, Integer> freq, int totalWords, int k) {
    TreeMap<Integer, Set<String>> treeMap = new TreeMap<>(Collections.reverseOrder());
    for (Map.Entry<String, Integer> e : freq.entrySet()) {
        treeMap.putIfAbsent(e.getValue(), new HashSet<>());
        treeMap.get(e.getValue()).add(e.getKey());
    }

    ArrayList<WordCount> topK = new ArrayList<>();
    for (Map.Entry<Integer, Set<String>> entry : treeMap.entrySet()) {
        for (String word : entry.getValue()) {
            topK.add(new WordCount(word, entry.getKey(), totalWords));
            if (topK.size() >= k) {
                break;
            }
        }
        if (topK.size() >= k) {
            break;
        }
    }

    return topK;
}
```

Map

Time Complexity:

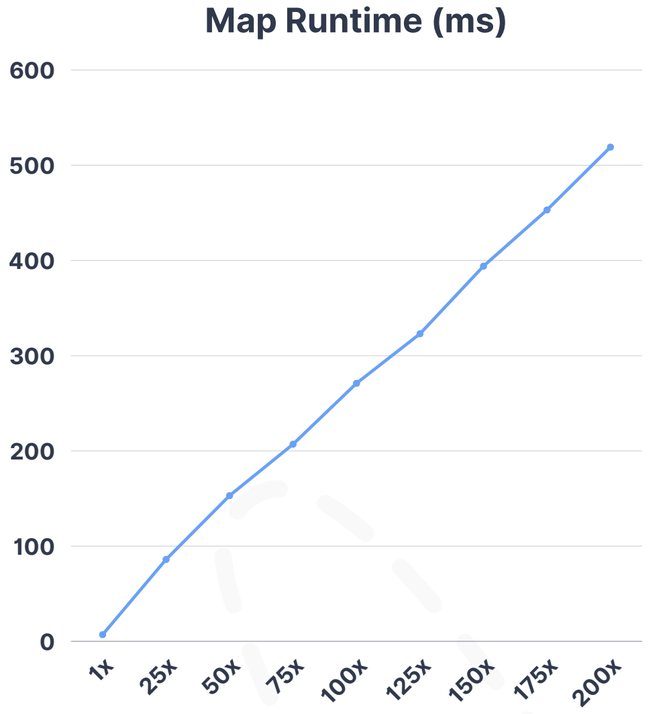
- O(N x M + Wlog(W))

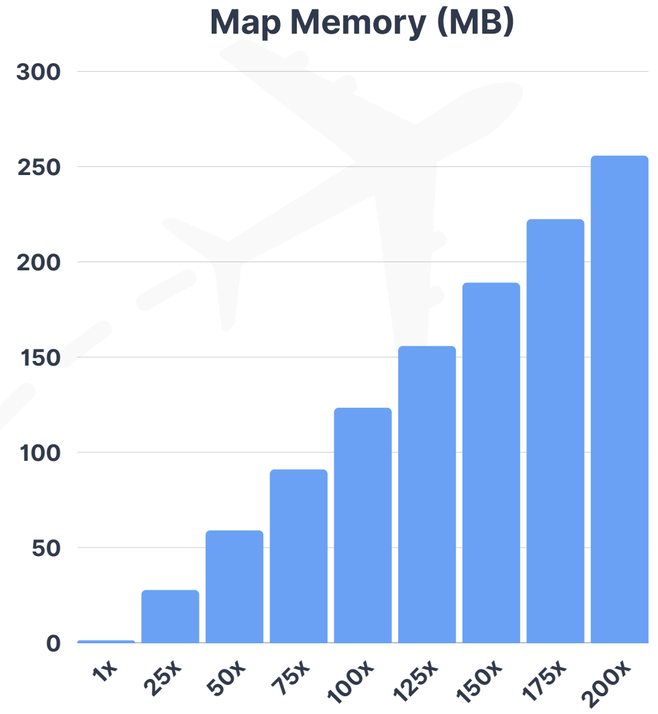
Space Complexity:

- O(W)

Where:

- N = Number of reviews
- M = Average words per review
- W = Unique words total





Empirical Results	1x	25x	50x	75x	100x	125x	150x	175x	200x
Runtime (ms)	7	86	153	207	271	323	394	453	519
Memory (MB)	1.274	27.701	58.995	91.012	123.409	155.806	189.08	222.429	255.779

Binary Search Tree

1. Grab reviews: get this airline's reviews; if none, return two empty top-10 lists.

2. Build counters: make two BSTs (one for good words, one for bad words).

3. Stream words: for each review, insert each token into the right BST (increments count if it already exists).

4. Keep top 10: in-order walk each BST and maintain a small min-heap of the biggest counts (no need to build a full list).

5. Return results: sort the kept words high→low and output topGood and topBad.

```
// Inner class: BST Node
private static class BSTNode {
    String word;
    int count;
    BSTNode left;
    BSTNode right;

    BSTNode(String word) {
        ...
    }
}

// Heap-based Top-K without building a full list
public List<WordCount> topK(int totalWords, int k) {
    PriorityQueue<WordCount> minHeap = new PriorityQueue<>(
        k, (a, b) -> Integer.compare(a.getCount(), b.getCount())
    );
    inOrderCollectTopK(root, minHeap, totalWords, k);

    ArrayList<WordCount> result = new ArrayList<>(minHeap);
    result.sort((a, b) -> Integer.compare(b.getCount(), a.getCount()));

    return result;
}

private void inOrderCollectTopK(BSTNode node,
                                PriorityQueue<WordCount> heap,
                                int totalWords,
                                int k) {
    if (node == null) return;

    inOrderCollectTopK(node.left, heap, totalWords, k);
    WordCount wc = new WordCount(node.word, node.count, totalWords);

    if (heap.size() < k) {
        heap.offer(wc);
    } else if (!heap.isEmpty() && wc.getCount() > heap.peek().getCount()) {
        heap.poll();
        heap.offer(wc);
    }

    inOrderCollectTopK(node.right, heap, totalWords, k);
}
```

Binary Search Tree

Time Complexity:

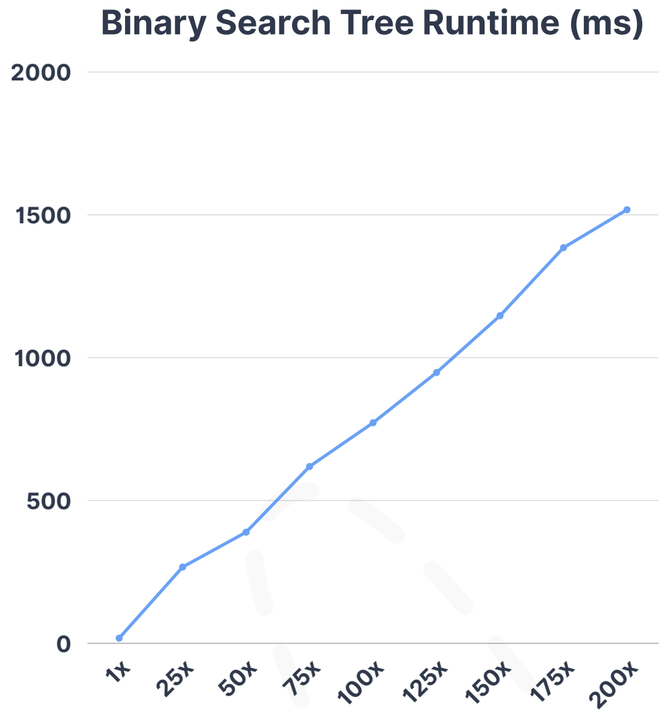
- O(N x M x log(N x M))

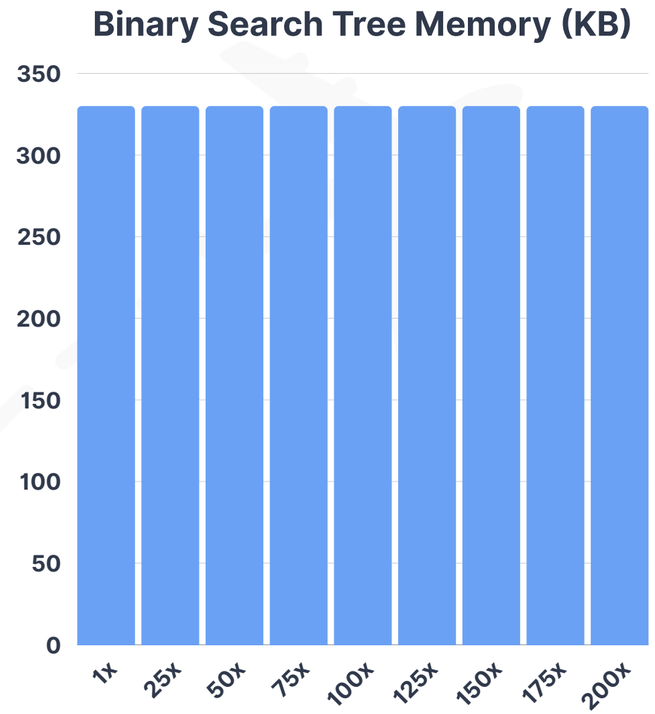
Space Complexity:

- O(N x M)

Where:

- N = Number of reviews
- M = Average words per review





Empirical Results	1x	25x	50x	75x	100x	125x	150x	175x	200x
Runtime (ms)	18	267	389	619	772	948	1147	1385	1518
Memory (MB)	0.33	0.33	0.33	0.33	0.33	0.33	0.33	0.33	0.33

Red-Black Tree

1. Grab reviews: get this airline's reviews; if none, return two empty top-10 lists.

2. Build counters: make two Red-Black Trees (one for good words, one for bad words).

3. Stream words: for each review, insert each token into the right tree (increments count if it already exists).

4. Keep top 10: in-order walk each tree and maintain a small min-heap of the biggest counts (no full list needed).

5. Return results: sort the kept words high→low and output topGood and topBad.

```
private static class WordFrequencyRBTree {
    private RBNode root;

    // Heap-based Top-K without building a full list
    public List<WordCount> topK(int totalWords, int k) {
        PriorityQueue<WordCount> minHeap =
            new PriorityQueue<>(k, (a, b) -> Integer.compare(a.getCount(), b.getCount()));
        inOrderCollectTopK(root, minHeap, totalWords, k);

        ArrayList<WordCount> result = new ArrayList<>(minHeap);
        result.sort((a, b) -> Integer.compare(b.getCount(), a.getCount()));
        return result;
    }

    private void inOrderCollectTopK(RBNode node,
                                    PriorityQueue<WordCount> heap,
                                    int totalWords,
                                    int k) {
        if (node == null) return;

        inOrderCollectTopK(node.left, heap, totalWords, k);
        WordCount wc = new WordCount(node.word, node.count, totalWords);

        if (heap.size() < k) {
            heap.offer(wc);
        } else if (!heap.isEmpty() && wc.getCount() > heap.peek().getCount()) {
            heap.poll();
            heap.offer(wc);
        }

        inOrderCollectTopK(node.right, heap, totalWords, k);
    }
}
```

Red-Black Tree

Time Complexity:

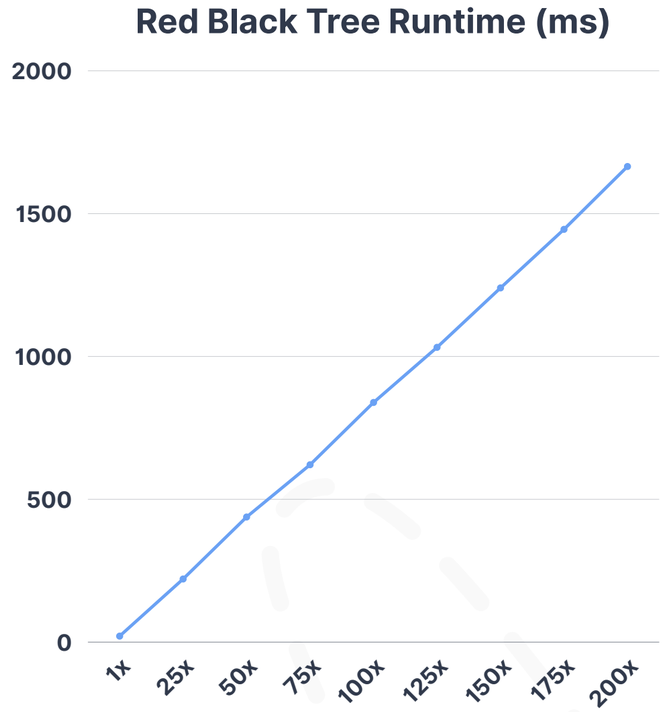
- O(N x M x log(W))

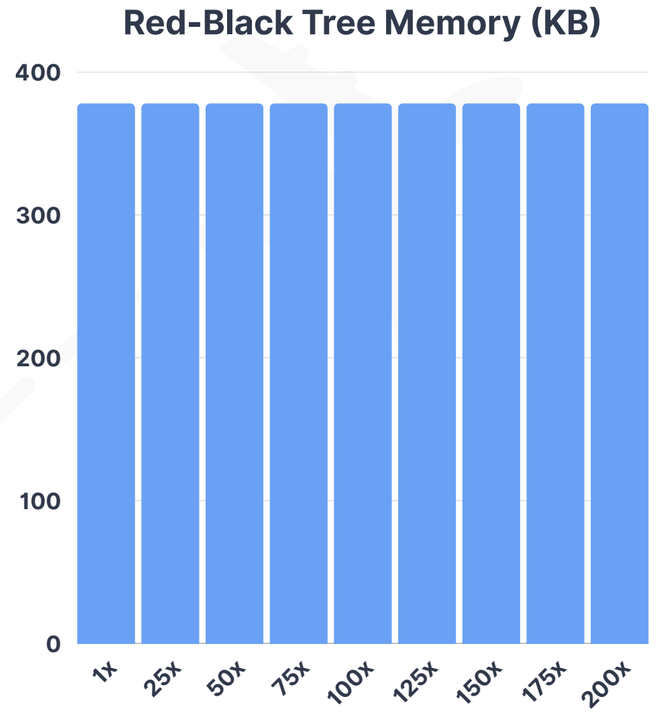
Space Complexity:

- O(W)

Where:

- N = Number of reviews
- M = Average words per review
- W = Unique Words





Empirical Results	1x	25x	50x	75x	100x	125x	150x	175x	200x
Runtime (ms)	21	221	438	621	839	1032	1240	1445	1665
Memory (MB)	0.378	0.378	0.378	0.378	0.378	0.378	0.378	0.378	0.378

AVL Tree

1. Collect reviews: get this airline's reviews; if none, return two empty top-10 lists.

2. Build counters: make two AVL trees (one for good words, one for bad words).

3. Stream words: for each review, send each token to the right tree and increase its count.

4. Pick top 10: do an in-order walk of each tree and keep the biggest counts using a small min-heap.

5. Return results: sort those kept words by count and output topGood and topBad.

```
private static class WordFrequencyAVL {
    private AVLNode root;

    // Get top K words by count using in-order traversal + min-heap
    public List<WordCount> topK(int totalWords, int k) {
        // Min-heap to keep top K elements
        PriorityQueue<WordCount> minHeap = new PriorityQueue<>(
            (a, b) -> Integer.compare(a.getCount(), b.getCount())
        );

        inOrderTraversal(root, totalWords, minHeap, k);

        // Convert heap to list (will be in ascending order, so reverse it)
        List<WordCount> result = new ArrayList<>(minHeap);
        result.sort((a, b) -> Integer.compare(b.getCount(), a.getCount()));

        return result;
    }

    private void inOrderTraversal(AVLNode node,
                                  int totalWords,
                                  PriorityQueue<WordCount> minHeap,
                                  int k) {
        if (node == null) return;

        inOrderTraversal(node.left, totalWords, minHeap, k);

        WordCount wc = new WordCount(node.word, node.count, totalWords);
        if (minHeap.size() < k) {
            minHeap.offer(wc);
        } else if (node.count > minHeap.peek().getCount()) {
            minHeap.poll();
            minHeap.offer(wc);
        }

        inOrderTraversal(node.right, totalWords, minHeap, k);
    }
}
```

AVL Tree

Time Complexity:

- O(N x M x log(W))

Space Complexity:

- O(W)

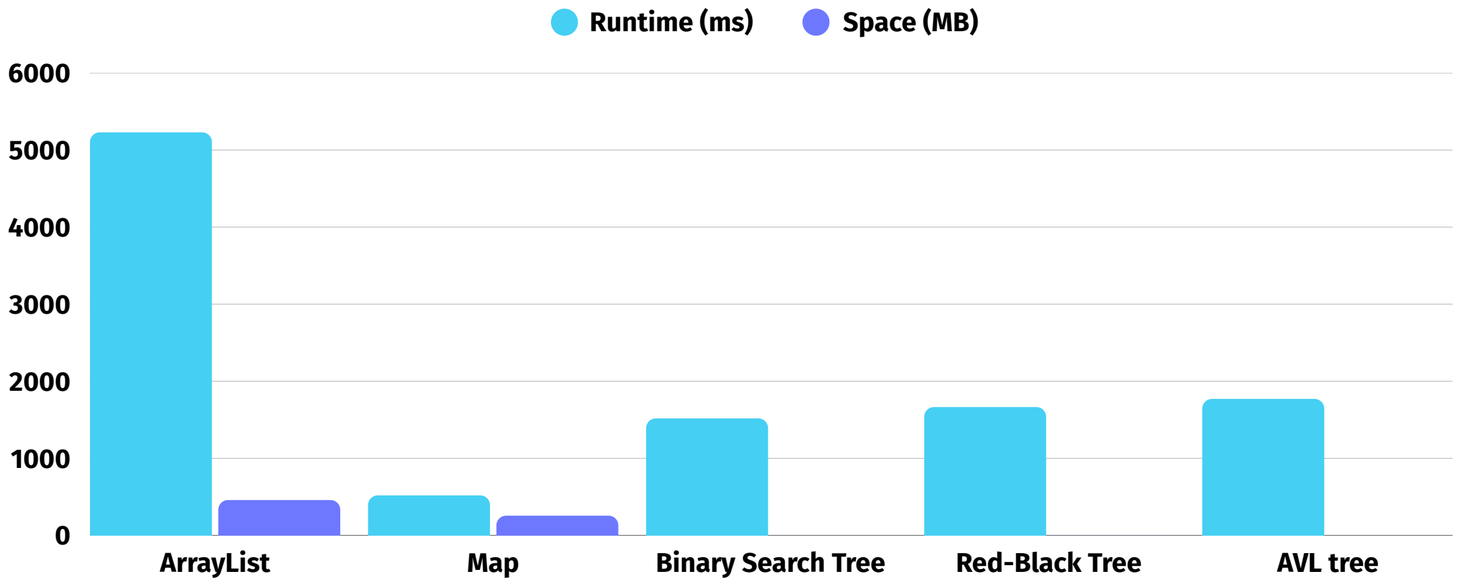
Where:

- N = Number of reviews
- M = Average words per review
- W = Unique Words

Empirical Results	1x	25x	50x	75x	100x	125x	150x	175x	200x
Runtime (ms)	26	259	467	681	903	1102	1355	1612	1771
Memory (MB)	0.33	0.33	0.33	0.33	0.33	0.33	0.33	0.33	0.33

Empirical Results: Implementation Advantage

Performance Metrics for Data Structures (ADT)



- For Top-K filtering (for 200x dataset)

Insights - Empirical Results

Trees uses least and constant space across datasets

Map is the clear winner on speed (runtime)

Recommendation:
Maps for static reviews, RBT for streaming/live-reviews

Where We Can Improve 😭

Live Counting and Alerts

- **Count words as they arrive, like a scoreboard that updates every second.**
- **Uses little memory and can trigger quick alerts when a word suddenly spikes**

```
Top counts now: "delay": 120, "seat": 11, "crew": 9
Alert! "delay" ≥ 10 × "seat"  → 120 ≥ 110 ✅
```

LSM Segments + Per-Shard Balanced Trees

- **Store by beginnings (“pro-”, “pre-”…).**
- **Super fast autocomplete and prefix Top-K.**

```
(pro)──(blem)    # "problem"
     └──(ject)   # "project"
(pro) alone      # word "pro"
(pre)──(y)       # "prey"
```

Log-Structured Storage + Per-Airline Trees

- **Write new data in fast little chunks, then flush them into disk after exceeding size limit in cache.**
- **Such as a small tree per airline so “top-K” and lookups stay quick even as data grows (merge outputs/counts from each tree).**

OOM CRASH ?
COOKED LA !